

本小节内容

课时 12 作业

课时 12 作业

Description

输入 3 4 5 6 7 9999 一串整数，9999 代表结束，通过尾插法新建链表，查找第二个位置的值并输出，在 2 个位置插入 99，输出为 3 99 4 5 6 7，删除第 4 个位置的值，打印输出为 3 99 4 5 6 7。

输出函数如下：

```
void PrintList(LinkList L)
{
    L = L->next;
    while (L != NULL)
    {
        printf("%3d", L->data); //打印当前结点数据

        L = L->next; //指向下一个结点
    }

    printf("\n");
}
```

针对双向链表，有时间的同学自己练习即可，这道题同样也可以用双向链表来实现一遍

Input

输入是 3 4 5 6 7 9999

Output

输出是

4

3 99 4 5 6 7

3 99 4 6 7

这道题考察的，和上课讲解的代码是基本是一致的，注意是链表的插入，删除。

答案如下：

```
#include <stdio.h>
#include <stdlib.h>

typedef int ElemType;
typedef struct LNode {
    ElemType data;
    struct LNode* next; //指向下一个结点
} LNode, * LinkList;

//头插法新建链表
LNode* CreatList1(LinkList& L) //list_head_insert
{
    LNode* s; int x;
    L = (LinkList)malloc(sizeof(LNode)); //带头结点的链表
    L->next = NULL; //L->data 里边没放东西
    scanf("%d", &x); //从标准输入读取数据
    //3 4 5 6 7 9999
    while (x != 9999) {
        s = (LNode*)malloc(sizeof(LNode)); //申请一个新空间给 s，强制类型转换
        s->data = x; //把读取到的值，给新空间中的 data 成员
        s->next = L->next; //让新结点的 next 指针指向链表的第一个元素(第一个放我们数据的元素)
        L->next = s; //让 s 作为第一个元素
        scanf("%d", &x); //读取标准输入
    }
    return L;
}

//尾插法新建链表
LinkList CreatList2(LinkList& L) //list_tail_insert
```

```

{
    int x;
    L = (LinkedList)malloc(sizeof(LNode)); //带头节点的链表
    LNode* s, *r = L; //LinkedList s, r=L; 也可以, r 代表链表表尾结点, 指向链表尾部
    //3 4 5 6 7 9999
    scanf("%d", &x);
    while (x != 9999) {
        s = (LNode*)malloc(sizeof(LNode));
        s->data = x;
        r->next = s; //让尾部结点指向新结点
        r = s; //r 指向新的表尾结点
        scanf("%d", &x);
    }
    r->next = NULL; //尾结点的 next 指针赋值为 NULL
    return L;
}

//打印链表
void PrintList(LinkedList L)
{
    L = L->next;
    while (L != NULL) //NULL 是为了代表一张空的藏宝图
    {
        printf("%3d", L->data); //打印当前结点数据
        L = L->next; //指向下一个结点
    }
    printf("\n");
}

//查找第几个结点的值
LinkedList GetElem(LinkedList L, int i)
{
    int j = 1;
    LinkedList p = L->next; //让 p 指向第一个结点
    if (0 == i)
    {
        return L; //i 是零就返回头结点
    }
    if (i < 1)
    {
        return NULL; //i 是负值就返回空
    }
    while (p && j < i)
    {
        p = p->next; //让 p 指向下一个结点
        j++;
    }
}

```

```

    }
    return p;
}
//按值查找
LinkedList LocateElem(LinkedList L, ElemType e)
{
    LinkedList p = L->next;
    while (p != NULL && p->data != e)
    {
        p = p->next;
    }
    return p;
}

```

//往第 i 个位置插入元素

```

bool ListFrontInsert(LinkedList L, int i, ElemType e)
{
    LinkedList p = GetElem(L, i - 1); //拿到要插入位置的前一个位置的地址值
    if (NULL == p)
    {
        return false; //i 不对
    }
    LinkedList s = (LinkedList)malloc(sizeof(LNode)); //给新结点申请空间
    s->data = e; //要插入的值放入对应空间
    s->next = p->next; //插入步骤
    p->next = s;
    return true;
}

```

//删除第 i 个位置的元素

```

bool ListDelete(LinkedList L, int i)
{
    LinkedList p = GetElem(L, i - 1); //查找删除位置的前驱节点
    if (NULL == p)
    {
        return false; //要删除的位置不存在
    }
    LinkedList q = p->next;
    if (NULL == q)
    {
        return false; //要删除的位置不存在
    }
    p->next = q->next; //断链
    free(q); //释放对应结点的空间
    q = NULL; //为了避免野指针
    return true;
}

```

```
}  
//《王道 C 督学营》课程  
int main()  
{  
    LinkList L;//链表头，是结构体指针类型  
    LinkList search;//用来存储拿到的某一个节点  
    CreatList2(L);//输入数据可以为 3 4 5 6 7 9999，尾插法  
    search = GetElem(L, 2);//查找链表第二个位置的元素值  
    if (search != NULL)  
    {  
        printf("%d\n", search->data);  
    }  
    ListFrontInsert(L, 2, 99);//新结点插入第 i 个位置  
    PrintList(L);//链表打印  
    ListDelete(L, 4);//删除第 4 个结点  
    PrintList(L);//打印注意格式  
    return 0;  
}
```